Walmart Capstone (Team 2) - PromoPilot Pipeline

A guide to every file in the repository and how to read the results of Steps 01-05

Repository: github.com/Xingyao0508/Capstone | Folder: Desktop/capstone/Pipeline

In one paragraph. The pipeline takes EverTrail's raw tables and builds one clean modeling table with **26,600 rows**, one per customer x historical campaign assignment. Each row holds what was known *before* the email was sent (features) and what happened in the *14 days after* (outcomes). On top of that table it produces business KPIs (Step 04) and the required Logistic Regression purchase-propensity model (Step 05). Steps 06-10 (incremental model, July scoring, promo cost, optimization, submission) are not built yet.

Headline results

Question	Answer	Where it comes from
Is the data clean enough to model?	Yes - 79 data-quality tests: 76 pass, 3 warnings, 0 failures.	outputs/tables/data_quality_summary.csv
How often does an assigned customer buy within 14 days?	13.0% of assignments.	historical_feature_outcome
Average NCM per assignment?	$17.56 (total $467,007.51).	kpi tables
Do the offers create extra profit? (randomized data)	PCT_10: $-1.08 (not significant); PCT_20: $-6.17 (significantly negative); FREE_SHIP: +$3.56 (best, but CI crosses 0).	incremental_ncm_exploration.csv
How good is the propensity model?	Test (CP07) ROC-AUC 0.655; top 10% of customers buy at 2.2x the average rate.	logistic_metrics.csv, logistic_test_decile_lift.csv

1. How the files link together

There is no single master file. The notebooks are linked through the files they **save**: each step reads the saved output of the step before it. Always run them in number order (01 -> 05). Re-running overwrites the previous results (same file names); the version committed to GitHub is the permanent record.

Step	Reads	Writes (main output)
01 Data validation	data/raw/*.csv	data/processed/clean/*.parquet + data-quality tables
02 Historical features	data/processed/clean/	data/processed/historical_features
03 14-day outcomes	clean tables + historical_features	historical_outcomes + historical_feature_outcome (the master table)
04 Business analysis	historical_feature_outcome	KPI tables + 3 charts
05 Logistic model	historical_feature_outcome + feature_schema	model file + metrics + predictions + 1 chart

How to run on your computer

- Make sure the 11 raw CSVs are in **Pipeline/data/raw/** (they are not on GitHub).
- Open each notebook in Jupyter / VS Code and choose **Run All**, in order 01, 02, 03, 04, 05.
- Or, in PowerShell from the Pipeline folder, run each with: `python -m jupyter nbconvert --to notebook --execute --inplace notebooks\01_data_validation.ipynb` (then 02 ... 05).

2. What every file does

Code and setup

File	Purpose
README.md	Project overview, run order, key definitions and assumptions.
requirements.txt	Python packages needed (pandas, numpy, scikit-learn, matplotlib, joblib, pyarrow, jupyter).
.gitignore	Keeps raw data, caches and secrets out of GitHub.
src/common.py	Shared folder paths, constants (14-day window, offer list, CP01-CP07) and loaders for raw/clean/processed data.
src/features.py	All feature logic for Step 02 in one function, build_features(). Written so Step 07 can reuse it with the July cutoff (policy_as_of_ts).
notebooks/01_data_validation.ipynb	Checks every raw table (shapes, keys, links between tables, missing values, duplicates, timing, money totals) and saves clean tables.
notebooks/02_build_historical_features.ipynb	Builds 43 'what we knew before sending' features per assignment and proves no future data leaked in.
notebooks/03_construct_14_day_outcomes.ipynb	Measures what happened in the 14 days after each send (purchase, revenue, cost, returns, NCM) and joins it to the features.
notebooks/04_business_analysis.ipynb	KPI tables and charts by offer, campaign, tenure/value and browsing-intent segment; estimates offer impact from randomized data.
notebooks/05_logistic_propensity_model.ipynb	Trains, tunes and tests the Logistic Regression purchase-propensity baseline; saves the model.

Data folder (data/)

File	What it contains
data/raw/*.csv	Untouched copies of the 11 supplied files. Never modified (checked by file fingerprint). Not on GitHub.
data/processed/clean/*.parquet	Step 01 clean tables: order_items without the 160 duplicate rows, orders with an is_completed flag, web sessions with an is_identified flag; other tables unchanged.
data/processed/clean/order_items_clean.csv	The de-duplicated order-item table in CSV form for easy inspection.
data/processed/historical_features.csv / .parquet	Step 02 output: 26,600 rows x 50 columns (7 ID/context + 43 features).
data/processed/historical_outcomes.csv / .parquet	Step 03 output: 26,600 rows of 14-day outcome columns.
data/processed/historical_feature_outcome.csv / .parquet	**The master table** = features + outcomes, one row per assignment. Steps 04, 05 and later steps all use this. (.csv opens in Excel; .parquet is the same data, faster for Python.)

Output folder (outputs/)

File	What it shows
tables/data_quality_summary.csv	One row per data-quality test: test name, table, number of failing rows, PASS/WARN/FAIL, note.
tables/schema_summary.csv	Every raw table's row/column count, column names and data types vs. expected.
tables/missingness_summary.csv	Columns with missing values, count and rate.

File	What it shows
tables/feature_schema.csv	List of every column in the master table: feature family and numeric/categorical type. Step 05 reads this to pick model inputs.
tables/outcome_schema.csv	List of the 14-day outcome columns (these are never allowed as model inputs).
tables/kpi_by_offer.csv	KPIs per offer type, split by assignment source (exploration vs business_rule).
tables/kpi_by_campaign.csv	KPIs per campaign CP01-CP07 and source - shows stability over time.
tables/kpi_by_tenure_value.csv	KPIs by customer tenure band x 90-day value band.
tables/kpi_by_behavior_segment.csv	KPIs by browsing-intent segment (high/medium/low) x offer.
tables/kpi_by_membership.csv	KPIs by membership x offer.
tables/incremental_ncm_exploration.csv	Extra NCM of each offer vs NO_OFFER with 95% confidence interval: randomized (exploration) estimate and naive business_rule comparison.
tables/incremental_ncm_by_segment_exploration.csv	Same randomized estimate inside each segment (intent, value, tenure, membership).
tables/logistic_validation_grid.csv	Model performance on CP06 for each regularization strength C tried.
tables/logistic_metrics.csv	Final model scores on CP06 (validation) and CP07 (test).
tables/logistic_test_predictions.csv	Each CP07 assignment with its actual purchase_14d and predicted purchase probability.
tables/logistic_test_decile_lift.csv	CP07 customers ranked into 10 groups by predicted probability: actual purchase rate and lift per group.
figures/04_incremental_ncm_by_offer.png	Bar chart: randomized extra NCM per offer, with confidence intervals.
figures/04_kpi_by_campaign.png	Conversion and NCM per assignment across campaigns.
figures/04_incremental_ncm_by_segment.png	Extra NCM per offer inside intent and value segments.
figures/05_logistic_calibration_lift.png	Model calibration and purchase rate by decile.
models/logistic_propensity_model.joblib	The trained model (pre-processing + Logistic Regression in one object), refit on CP01-CP07, ready for July scoring in Step 07.
models/logistic_model_features.json	Exact input features, chosen C, train/validation/test split, and encoded feature names.

3. Reading the results, step by step

Step 01 - Data quality

Open **data_quality_summary.csv**. Every row is a test. **PASS** = no problem rows; **WARN** = a known, handled issue; **FAIL** = something broken (the notebook stops if any FAIL exists). Current result: 79 tests, 76 PASS, 3 WARN, 0 FAIL. The three warnings:

Test	Rows	What it means / how it is handled
exact_dup_rows_order_items (removed)	160	Exact duplicate item lines. Removed before any totals; afterwards item totals match order totals within $0.05.
anonymous web sessions (kept raw, excluded from features)	220	Sessions with no customer_id (identity not stitched). Kept in raw data, excluded from customer features.
refund <= merchandise_gross_revenue	5	Refund larger than merchandise revenue (but not larger than net revenue, so likely includes shipping). Kept.

Other facts confirmed: cancelled orders (102) have all money fields = 0 and never count as purchases; organic pre-period orders all happen before the first campaign; every campaign order matches exactly one assignment and falls inside its 14-day window; July (CP08) customers have no past campaign assignments.

Step 02 - Features (what we knew before sending)

Each row of **historical_features** is one assignment. The 43 features fall in six families. Rule: only events strictly **before send_ts** are used; '30d' means the 30 days before the send.

Family	Examples	How to read
Profile (8)	customer_tenure_days, customer_tier, is_member, region	Who the customer is. Tenure is measured at the send date.
Purchase (13)	prior_order_count, orders_90d, net_revenue_90d, historical_ncm_90d, prior_offer_redemption_rate	Past buying. 0 = no orders in the window; days_since_last_order blank = never ordered.
Returns (3)	returns_90d, refund_amount_90d, historical_return_rate	Return rate = returned past orders / past orders (blank if no past orders).
Web (8)	sessions_30d, cart_add_sessions_30d, cart_add_rate_30d, top_category_viewed_30d	Recent browsing intent.
Email (7)	prior_emails_delivered, historical_open_rate, days_since_last_email_click	Engagement with past campaign emails only (never the current one).
Product (4)	units_purchased_90d, distinct_categories_90d, top_purchase_category_90d	What they bought in the last 90 days.

Blank values are intentional: a rate or 'days since' is blank when it cannot be defined (e.g. no prior emails). The model fills them in later and adds a 'was missing' flag. The notebook also proves no leakage two ways: the latest event used is always before send_ts, and injecting fake future events changes no feature.

Step 03 - 14-day outcomes

Column	Meaning
purchase_14d	1 if the customer made at least one completed order for this campaign within 14 days after send_ts, else 0.
order_count_14d / net_revenue_14d / cogs_14d / shipping_cost_14d	Totals over those eligible orders (0 for non-buyers).
offer_redeemed_14d	1 if the assigned offer was actually used.

Column	Meaning
promo_cost_14d	Realized promotion cost: the discount for PCT_10/PCT_20; the waived $5.99 shipping fee for FREE_SHIP (inferred, flagged by promo_cost_inferred=1); 0 for NO_OFFER.
refund_amount_applicable	Refunds on those orders whose return happened **inside** the same 14-day window.
ncm_14d	**Net contribution margin** = net revenue - COGS - shipping cost - applicable refunds. The main money outcome.
aov_14d	Average order value = net revenue / orders, buyers only (blank for non-buyers).
post_window_refund_amount / ncm_14d_all_returns	Refunds that came after day 14, and NCM if those were also subtracted - for sensitivity checks.

Important assumption: returns arrive 3-31 days after purchase, so most refunds on campaign orders land after the 14-day window: $83,776 after vs $15,402 inside. The main NCM counts only the inside ones; ncm_14d_all_returns shows the stricter view. The team should agree which rule to use (one setting in notebook 03).

Step 04 - Business analysis

All KPI tables share the same columns:

Column	Definition
assignments	Number of customer-campaign rows in the group.
conversion	Share of assignments with purchase_14d = 1.
aov	Net revenue / number of orders.
redemption_rate	Offers redeemed / offers sent (blank for NO_OFFER).
promo_spend, promo_spend_per_assignment	Total realized promo cost, and per assignment.
ncm_total, ncm_per_assignment	Total NCM, and average NCM per assignment (the fairest comparison across group sizes).
ncm_all_returns_per_assignment	Same, subtracting all refunds including after day 14.

The key reading rule: exploration vs business_rule

Historical offers came from two sources. **Exploration** assignments were randomized, so the four offer groups contain similar customers - differences between them are caused by the offer. **Business_rule** assignments were targeted: EverTrail gave NO_OFFER to its strongest customers (more past orders, higher past NCM, more browsing). Comparing offers inside business_rule therefore mixes the offer's effect with who received it.

Offer	Conversion (exploration)	NCM / assignment (exploration)	Conversion (business_rule)	NCM / assignment (business_rule)
NO_OFFER	11.3%	$16.21	19.3%	$30.61
PCT_10	12.1%	$15.13	11.2%	$11.00
PCT_20	12.4%	$10.03	10.2%	$7.17
FREE_SHIP	13.1%	$19.77	12.0%	$25.26

Notice NO_OFFER looks best under business_rule (19.3% conversion) but not under exploration (11.3%) - that is the targeting bias.

Incremental NCM (does the offer create extra profit?)

incremental_ncm_exploration.csv answers this using randomized rows only: average NCM of the offer group minus average NCM of the NO_OFFER group. **ci_low / ci_high** give the 95% confidence interval: if it contains 0, the

effect is not clearly different from no effect. Rows labelled business_rule_naive are shown only to illustrate the bias - do not use them for decisions.

Offer	Source	Extra conversion	Extra NCM / assignment	95% CI	Reading
PCT_10	exploration	+0.8%	-1.08	-5.12 to +2.96	not clear
PCT_20	exploration	+1.1%	-6.17	-9.65 to -2.70	hurts profit
FREE_SHIP	exploration	+1.8%	+3.56	-0.89 to +8.01	not clear
PCT_10	business_rule (naive)	-8.2%	-19.61	-22.26 to -16.96	biased - ignore
PCT_20	business_rule (naive)	-9.1%	-23.45	-26.37 to -20.52	biased - ignore
FREE_SHIP	business_rule (naive)	-7.4%	-5.36	-9.11 to -1.60	biased - ignore

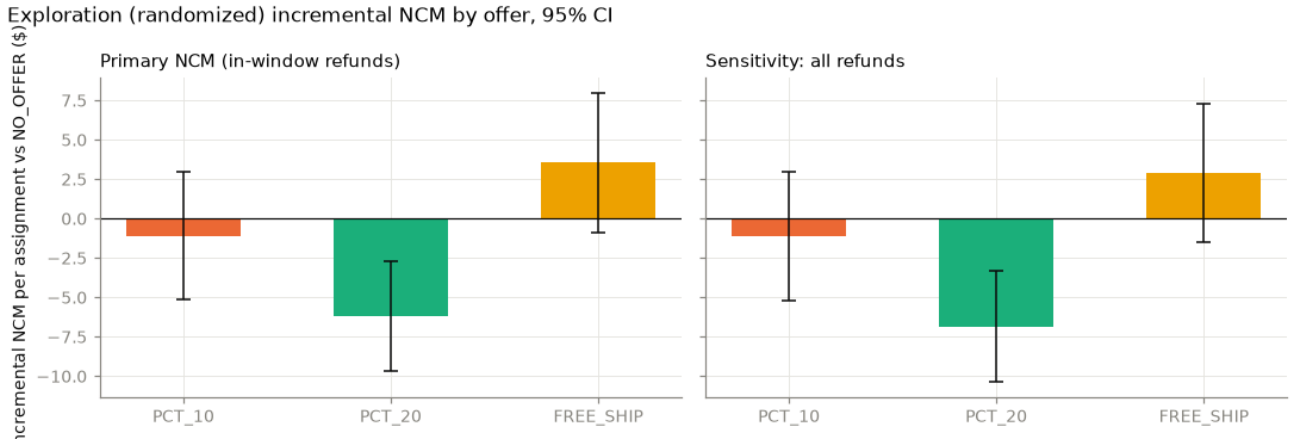


Figure 1. Randomized extra NCM per assignment vs NO_OFFER. Bars = estimate, whiskers = 95% CI. Left: main NCM; right: counting all refunds. PCT_20 is clearly negative: the 20% discount costs more margin than the extra purchases bring in.

How to read it: offers lift conversion only slightly (+1-2 points) but discounts give away margin on purchases that would have happened anyway. FREE_SHIP is cheapest (fee waived on few orders) and comes out best.

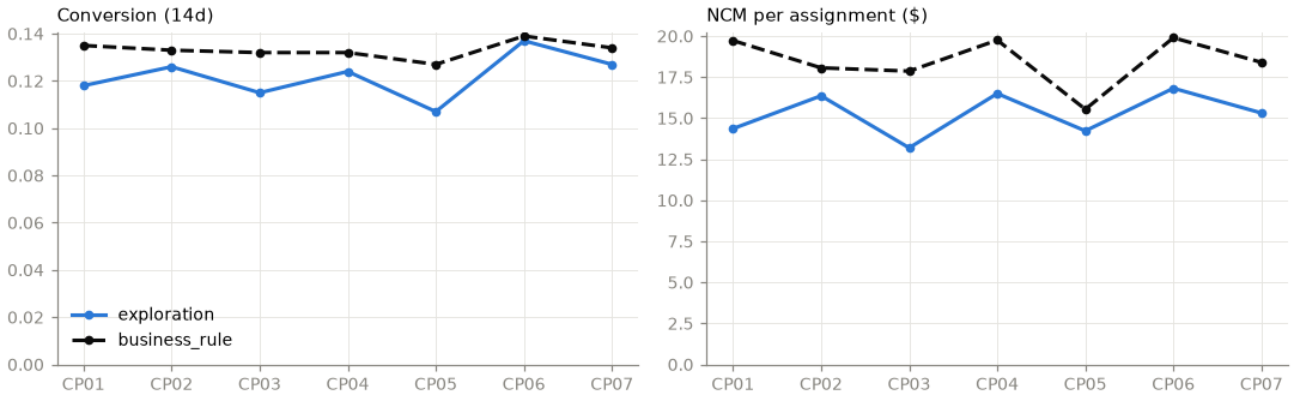


Figure 2. Conversion and NCM per assignment by campaign. Stable across CP01-CP07 (no campaign is an outlier), which supports training on earlier campaigns and testing on later ones.

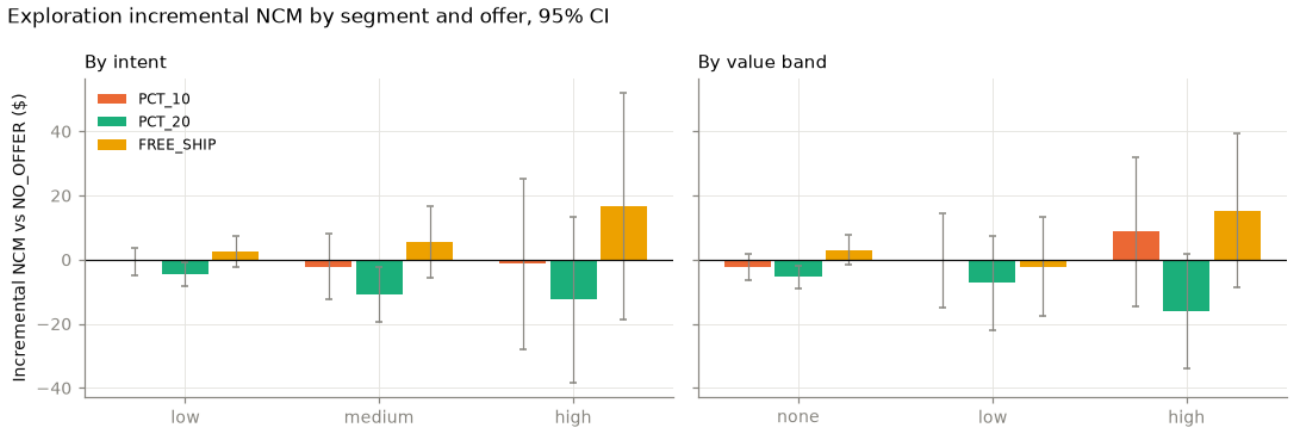


Figure 3. Randomized extra NCM by browsing-intent segment (left) and 90-day value band (right). Wide whiskers = small groups, so treat segment differences as hints for Step 06, not conclusions.

Segment definitions

- **Intent:** high = added to cart in the last 30 days; medium = browsed in the last 30 days without cart, or last visit 31-90 days ago; low = no visit in 90 days.
- **Value band:** none = no completed order in last 90 days; low/high = below/above the median 90-day NCM among recent buyers.
- **Tenure band:** under 1 year, 1-2 years, 2+ years since signup.

Patterns worth noting: FREE_SHIP is positive in most segments and largest for high-intent and high-value customers; PCT_20 is negative in every segment; PCT_10 is near zero or negative except for new (under 1 year) and high-value customers. These are the kinds of differences the Step 06 T-Learner will try to exploit customer by customer.

Step 05 - Logistic purchase-propensity model

What it predicts: the probability a customer buys within 14 days (purchase_14d), using only the 43 pre-send features. It deliberately ignores which offer the customer got - it ranks *who is likely to buy*, not *who an offer changes*. This is the required baseline: later, baseline policy #2 sends PCT_10 to the highest-probability customers.

How it was tested: trained on CP01-CP05 (19,000 rows), tuned on CP06 (3,800), and scored once on CP07 (3,800), which the model never saw - mimicking predicting a future campaign. The final saved model is refit on all CP01-CP07.

Choosing C (logistic_validation_grid.csv)

C (regularization)	ROC-AUC	PR-AUC	Log loss	Brier
0.01	0.6862	0.2652	0.3756	0.1124
0.1	0.6851	0.2629	0.3755	0.1123
1.0	0.6837	0.2590	0.3759	0.1124
10.0	0.6841	0.2601	0.3758	0.1124

Smaller C = simpler model. All values perform almost the same; C = 0.01 had the best validation ROC-AUC and was selected.

Final scores (logistic_metrics.csv)

Metric	CP06 valid	CP07 test	Plain-English meaning
ROC-AUC	0.686	0.655	Chance a random buyer is ranked above a random non-buyer. 0.5 = coin flip, 1.0 = perfect. ~0.65-0.69 = modest but real ranking power.
PR-AUC	0.265	0.231	Precision across all cut-offs. Compare to the base rate (13.2%): about 1.7-1.9x better than random.
Log loss	0.376	0.373	Penalty for confident wrong probabilities. Lower is better.
Brier score	0.112	0.110	Average squared error of the probabilities. Lower is better.
Mean prediction vs actual	0.127 / 0.138	0.125 / 0.132	Close values = probabilities are on the right scale.

Precision/recall/F1 rows: at the default 0.50 threshold the model predicts nobody buys (every probability is below 0.5 because only ~13% buy), so precision/recall are 0 - this is expected and not an error. Using a threshold of 0.17 (chosen on CP06), on CP07 it flags 819 customers, of whom 193 bought (precision 23.6%), catching 38.5% of all buyers. For targeting we use the *ranking*, not a threshold.

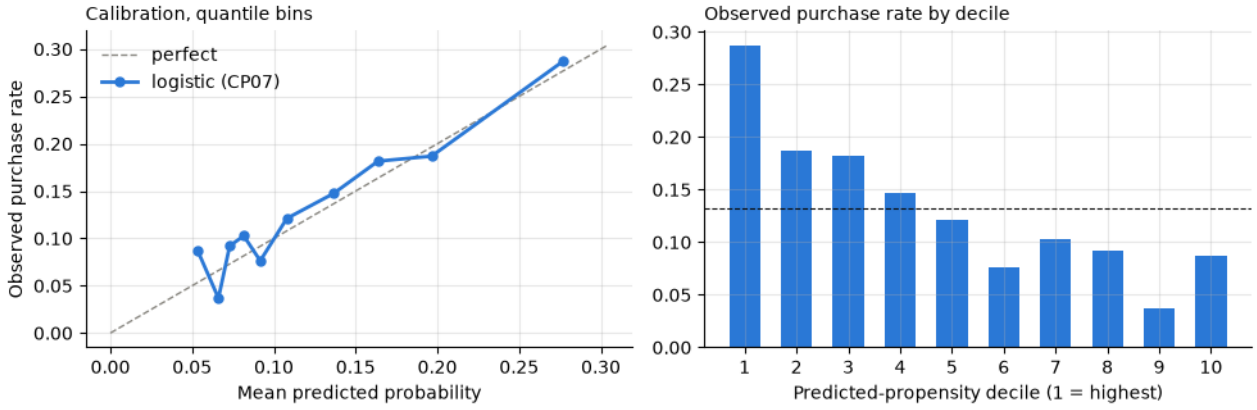


Figure 4. Left: calibration - points near the dashed line mean a predicted 20% really buys about 20% of the time. Right: actual CP07 purchase rate per predicted decile (1 = highest probability); the dashed line is the average.

Decile lift (logistic_test_decile_lift.csv)

Decile	Customers	Actual purchase rate	Avg predicted	Lift vs average
1	380	28.7%	27.7%	2.18x
2	380	18.7%	19.6%	1.42x
3	380	18.2%	16.4%	1.38x
4	380	14.7%	13.6%	1.12x
5	380	12.1%	10.8%	0.92x
6	380	7.6%	9.2%	0.58x
7	380	10.3%	8.2%	0.78x
8	380	9.2%	7.3%	0.70x
9	380	3.7%	6.6%	0.28x
10	380	8.7%	5.3%	0.66x

Read it as: the 10% of customers the model rates highest bought at 28.7%, 2.2x the average. The top three deciles are clearly above average; the bottom deciles are below. The strongest predictor is customer tier (Gold up, Standard down), followed by recent browsing and past email/purchase behaviour.

4. Important caveats before using these results

- **Propensity is not profit.** The model finds likely buyers; many of them buy anyway, so sending them discounts can lose money (Step 04 shows this). Step 06 will estimate incremental NCM per customer.
- **Use exploration rows for cause-and-effect.** Business_rule comparisons are biased by targeting.
- **Return timing changes NCM.** Most refunds land after day 14; the team must confirm the in-window rule or switch RETURN_RULE in notebook 03.
- **FREE_SHIP promo cost is inferred** from the $5.99-under-$75 shipping rule seen in the data (it held for every non-FREE_SHIP order).
- **Profile fields have no history** (tier, membership etc. are current values), treated as known at decision time.
- **July customers are new to campaigns**: none appear in historical assignments, so their email-history features will be 0/blank in Step 07.
- **Segment estimates are noisy** - wide confidence intervals in small groups.

5. Glossary

Term	Meaning
Assignment	One customer receiving one action in one campaign (row of historical_assignments).
send_ts	When the campaign email was sent; the cutoff for features.
NCM	Net contribution margin: net revenue - COGS - shipping cost - applicable refunds.
Incremental NCM	NCM with the offer minus NCM the same kind of customer generates with NO_OFFER.
Exploration / business_rule	Randomized test assignments / EverTrail's normal targeted assignments.
Leakage	Accidentally using information from after the decision; makes models look better than they are.
Parquet	Compressed table file format; same content as the CSV, faster to load in Python.
joblib	File format for saving a trained Python model.
CP01-CP07 / CP08_POLICY	Historical campaigns (Jan-May 2025) / the July campaign to be scored.